

You Only Look Once: Unified, Real-Time Object Detection

Contents

- Introduction
- Unified Detection
 - Network Design
 - Inference Process / Training Process
- Experiments
- Conclusion

YOLO: Unified, Real-Time Object Detection

You Only Look Once: Unified, Real-Time Object Detection

Joseph Redmon*, Santosh Divvala*[†], Ross Girshick[¶], Ali Farhadi*[‡]

University of Washington*, Allen Institute for AI[†], Facebook AI Research[¶]

<http://pjreddie.com/yolo/>

Abstract

We present YOLO, a new approach to object detection. Prior work on object detection repurposes classifiers to perform detection. Instead, we frame object detection as a regression problem to spatially separated bounding boxes and associated class probabilities. A single neural network predicts bounding boxes and class probabilities directly from full images in one evaluation. Since the whole detection pipeline is a single network, it can be optimized end-to-end directly on detection performance.

Our unified architecture is extremely fast. Our base YOLO model processes images in real-time at 45 frames per second. A smaller version of the network, Fast YOLO, processes an astounding 155 frames per second while still achieving double the mAP of other real-time detectors. Compared to state-of-the-art detection systems, YOLO makes more localization errors but is less likely to predict false positives on background. Finally, YOLO learns very general representations of objects. It outperforms other detection methods, including DPM and R-CNN, when generalizing from natural images to other domains like artwork.

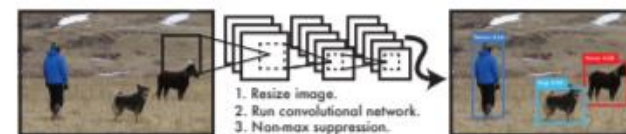
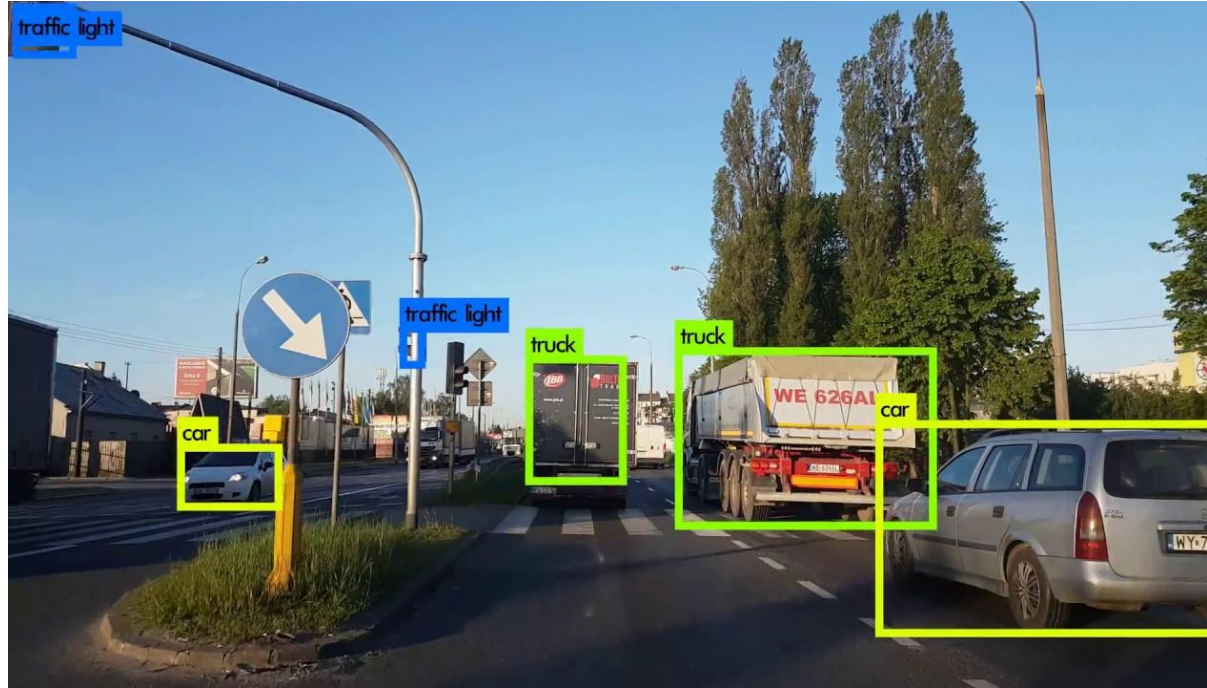


Figure 1: The YOLO Detection System. Processing images with YOLO is simple and straightforward. Our system (1) resizes the input image to 448×448 , (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model's confidence.

methods to first generate potential bounding boxes in an image and then run a classifier on these proposed boxes. After classification, post-processing is used to refine the bounding boxes, eliminate duplicate detections, and rescore the boxes based on other objects in the scene [13]. These complex pipelines are slow and hard to optimize because each individual component must be trained separately.

We reframe object detection as a single regression problem, straight from image pixels to bounding box coordinates and class probabilities. Using our system, you only look once (YOLO) at an image to predict what objects are present and where they are.

Introduction



- 사람은 이미지를 봤을 때, 이미지 내부에 위치하는 객체들의 디테일을 한 번에 파악할 수 있음
 - e.g.) 사진 속 객체가 무엇인지, 어디에 위치하는지 등의 정보를 의식하지 않고도 쉽게 파악 가능 → “운전 가능”
- But, R-CNN과 같은 이전 Object Detection 시스템은 복잡한 처리 과정으로 인간의 시각 시스템 모방하기 역부족
- YOLO는 이미지 내 bounding box와 class probability를 하나의 regression 문제로 엮어,
이미지를 한 번 보는 것만으로 이미지 내에 위치한 객체의 종류와 위치 정보를 추측할 수 있는 모델 !

Introduction

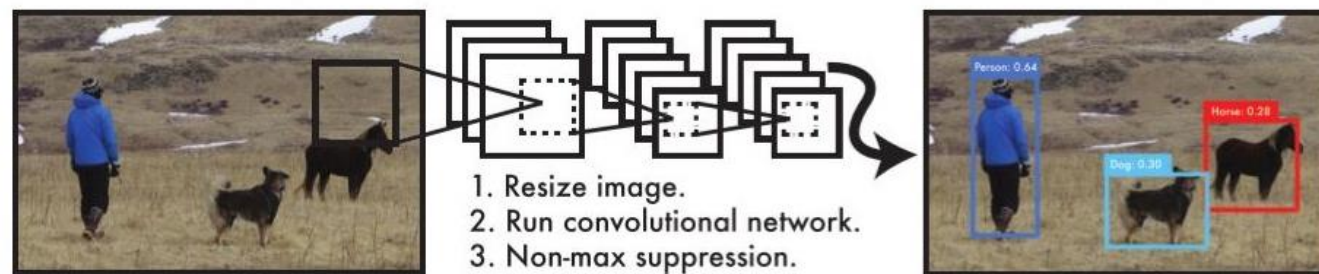


Figure 1: The YOLO Detection System. Processing images with YOLO is simple and straightforward. Our system (1) resizes the input image to 448×448 , (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model's confidence.

- Pros:
 - 간단한 처리과정으로 인해 **빠른 속도** + 기존 Real-Time Detection 시스템들과 비교해 **2배** 가량 높은 mAP
 - 이미지 전체를 한 번에 바라보기 때문에 Class에 대한 **맥락 이해**가 높고, 이 덕분에 Background error가 낮음
 - 이미지 전체를 보며 객체의 **일반화된 특징**을 학습 → “사진 속 의자의 모습을 학습한 후, 정물화 속 의자 추측 가능”
- Cons: 다른 객체 인식 모델에 비해 상대적으로 낮은 정확도 (*esp. 크기가 작은 객체 검출 시*)

Unified Detection

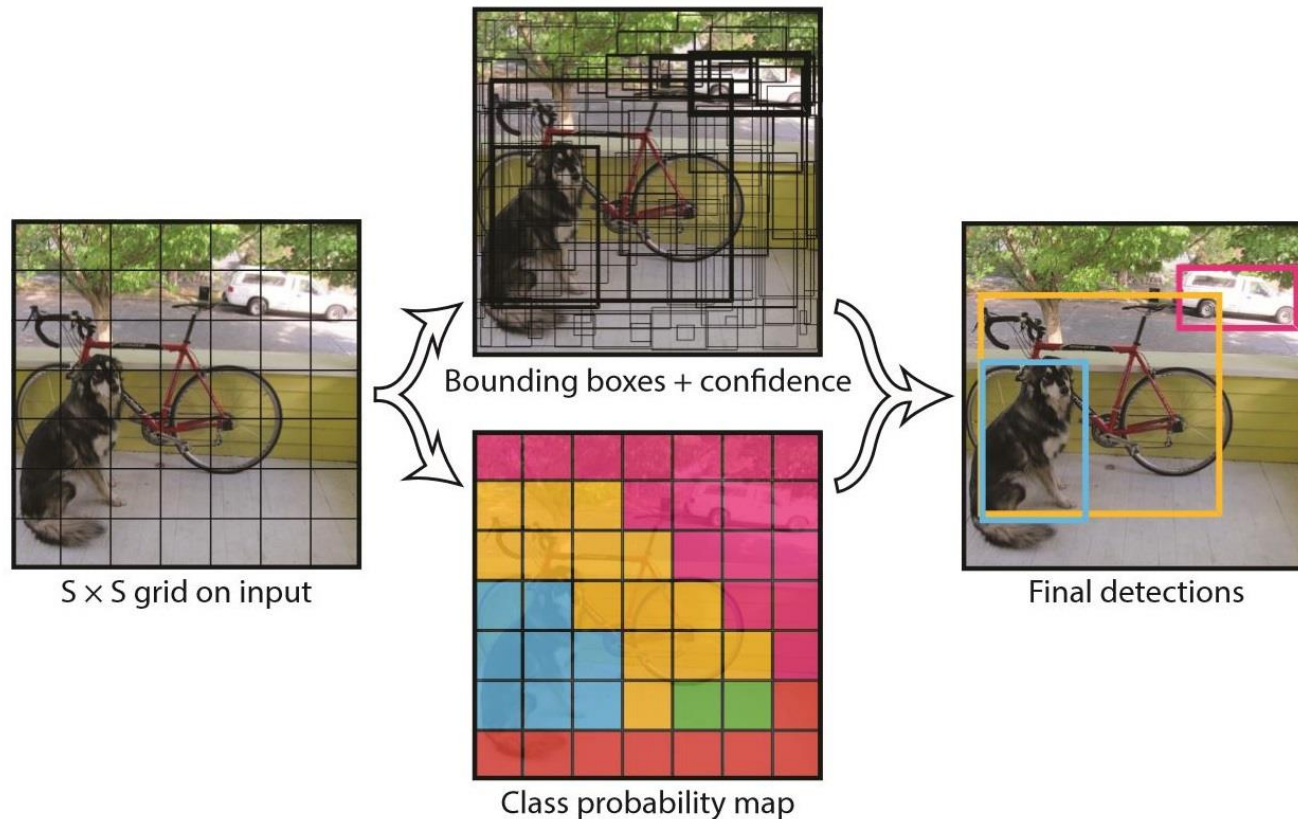
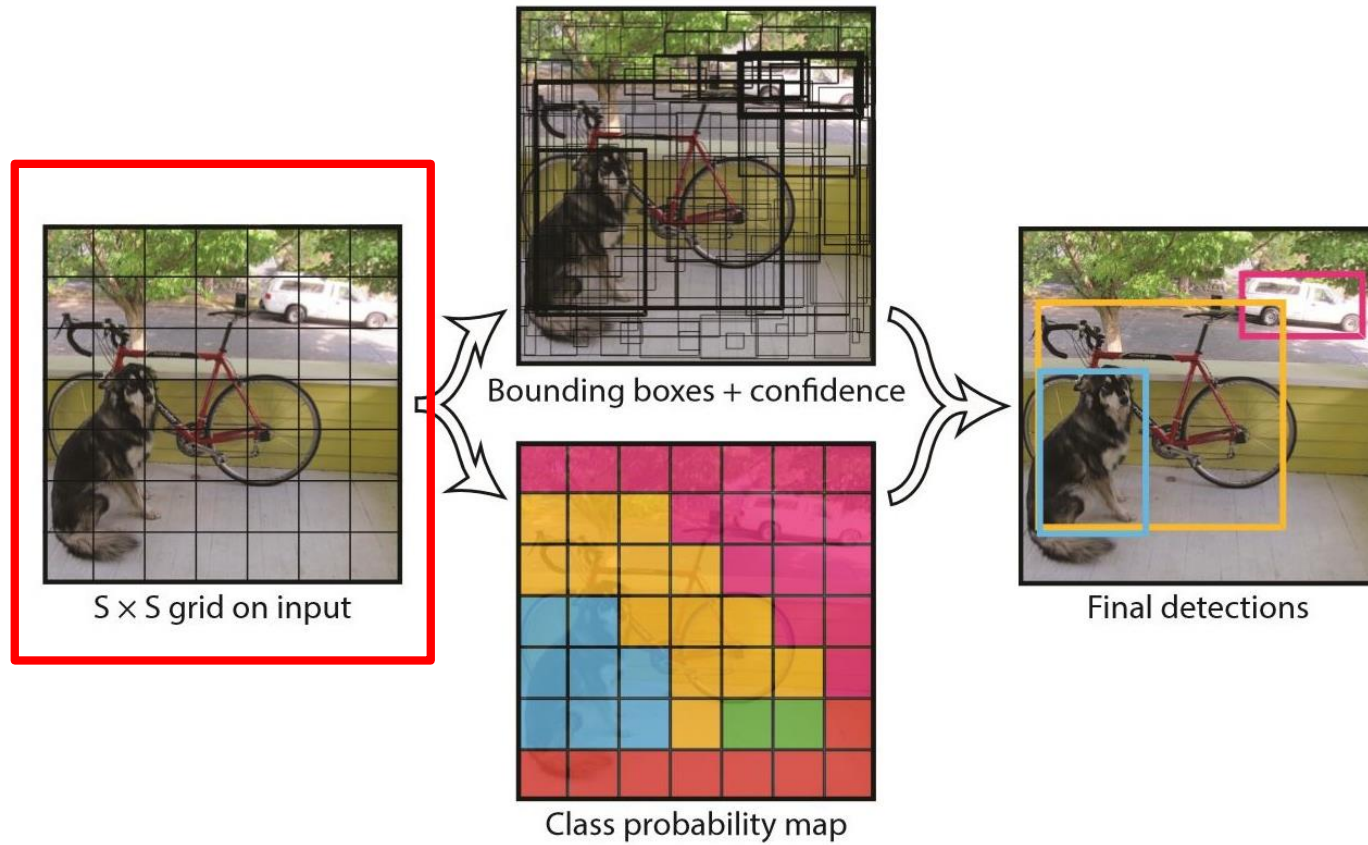


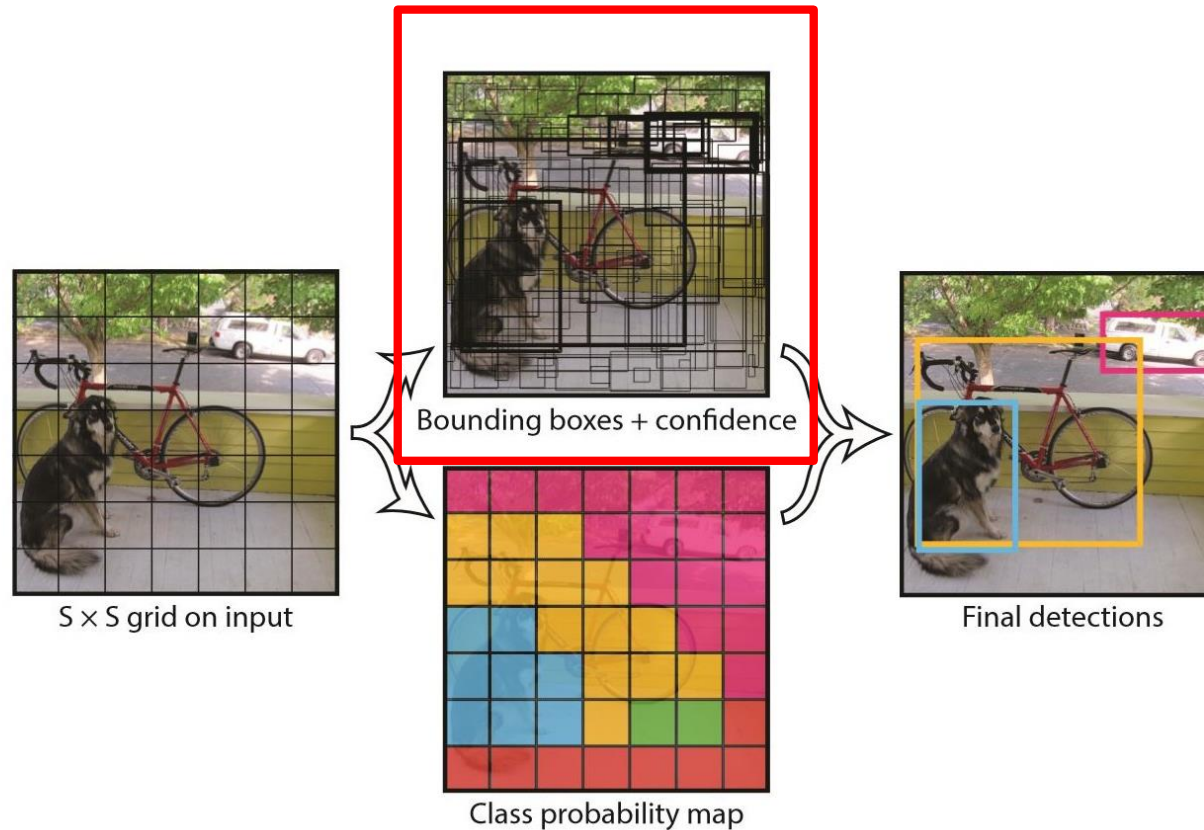
Figure 2: The Model. Our system models detection as a regression problem. It divides the image into an $S \times S$ grid and for each grid cell predicts B bounding boxes, confidence for those boxes, and C class probabilities. These predictions are encoded as an $S \times S \times (B * 5 + C)$ tensor.

Unified Detection



1. Input Image를 $S \times S$ 개의 Grid로 나눔 (In the paper, $S = 7$)

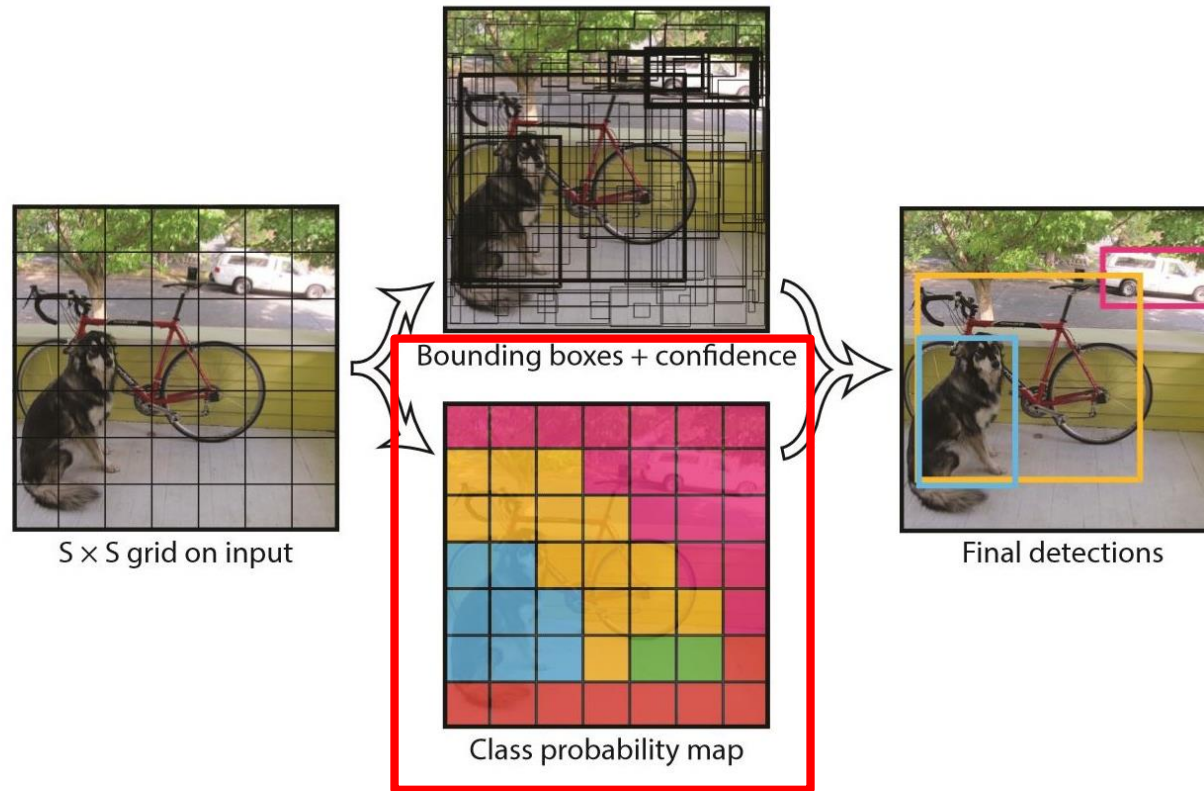
Unified Detection



2. 각각의 Grid cell은 B개의 Bounding box와 각 bounding box에 대한 confidence score를 가짐 [**Confidence Score: $\Pr(\text{Object}) * \text{IOU}_{\text{truthpred}}$**]

cf) In the paper, $B = 2 \rightarrow$ totally 98 bounding boxes / cell에 object가 없으면 confidence score는 0이 됨

Unified Detection



3. 각각의 Grid cell은 C개의 Conditional Class Probability를 가짐

[Conditional Class Probability: $\Pr(\text{Class } i \mid \text{Object})$]

*cf) In the paper, $C = 20 \rightarrow$ totally $49 * 20 = 980$ class probabilities*

Unified Detection

4. 각각의 Bounding box는 x, y 좌표, w, h 그리고

confidence를 지님

(x, y) : Bounding box의 중심점을 의미하며, grid cell

범위에 대한 상대 값

e.g.) x 가 grid cell 가장 왼쪽, y 가 grid cell 중간에

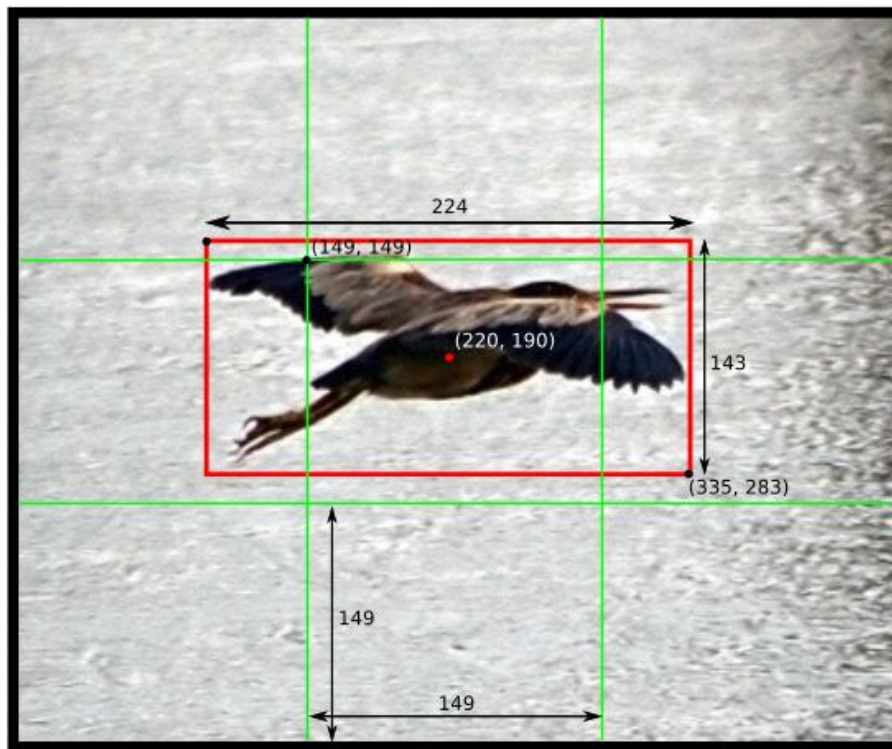
있다면 $x=0, y=0.5$

(w, h) : 전체 이미지의 width, height에 대한 상대 값

e.g.) Bounding box의 width가 전체 이미지 width의

절반이라면 $w=0.5$

(0, 0)



$$x = (220 - 149) / 149 = 0.48$$

$$y = (190 - 149) / 149 = 0.28$$

$$w = 224 / 448 = 0.50$$

$$h = 143 / 448 = 0.32$$

(447, 447)

Unified Detection

Test Time에는 Conditional Class Probability와 Bounding box의 confidence score를 곱해,
Class-Specific Confidence Score를 계산 !

$$\begin{aligned} \textit{ClassSpecificConfidenceScore} &= \textit{ConditionalClassProbability} * \textit{ConfidenceScore} \\ &= Pr(\textit{Class}i|\textit{Object}) * Pr(\textit{Object}) * IOU_{pred}^{truth} \\ &= Pr(\textit{Class}i) * IOU_{pred}^{truth} \end{aligned}$$

Network Design

- YOLO의 기본 Network Architecture는 GoogLeNet을 기반으로 함
- 24개의 Convolutional Layer + 2개의 Fully Connected Layer
- *Fast YOLO에서는 24개가 아닌 9개의 Convolutional Layer 사용*

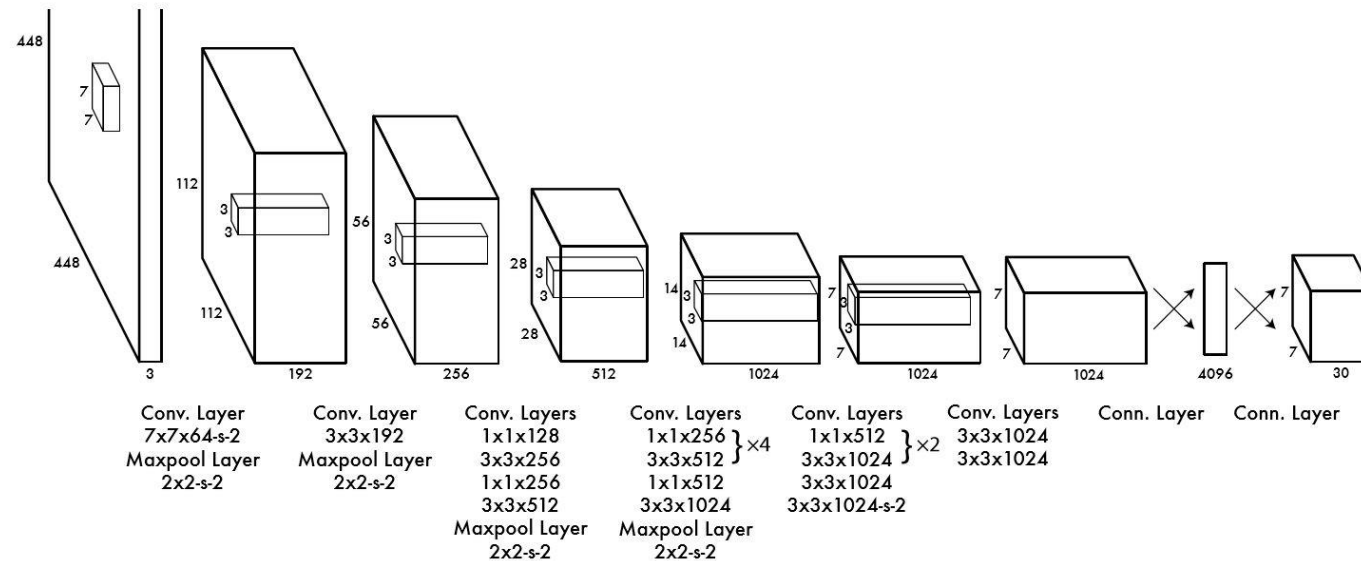
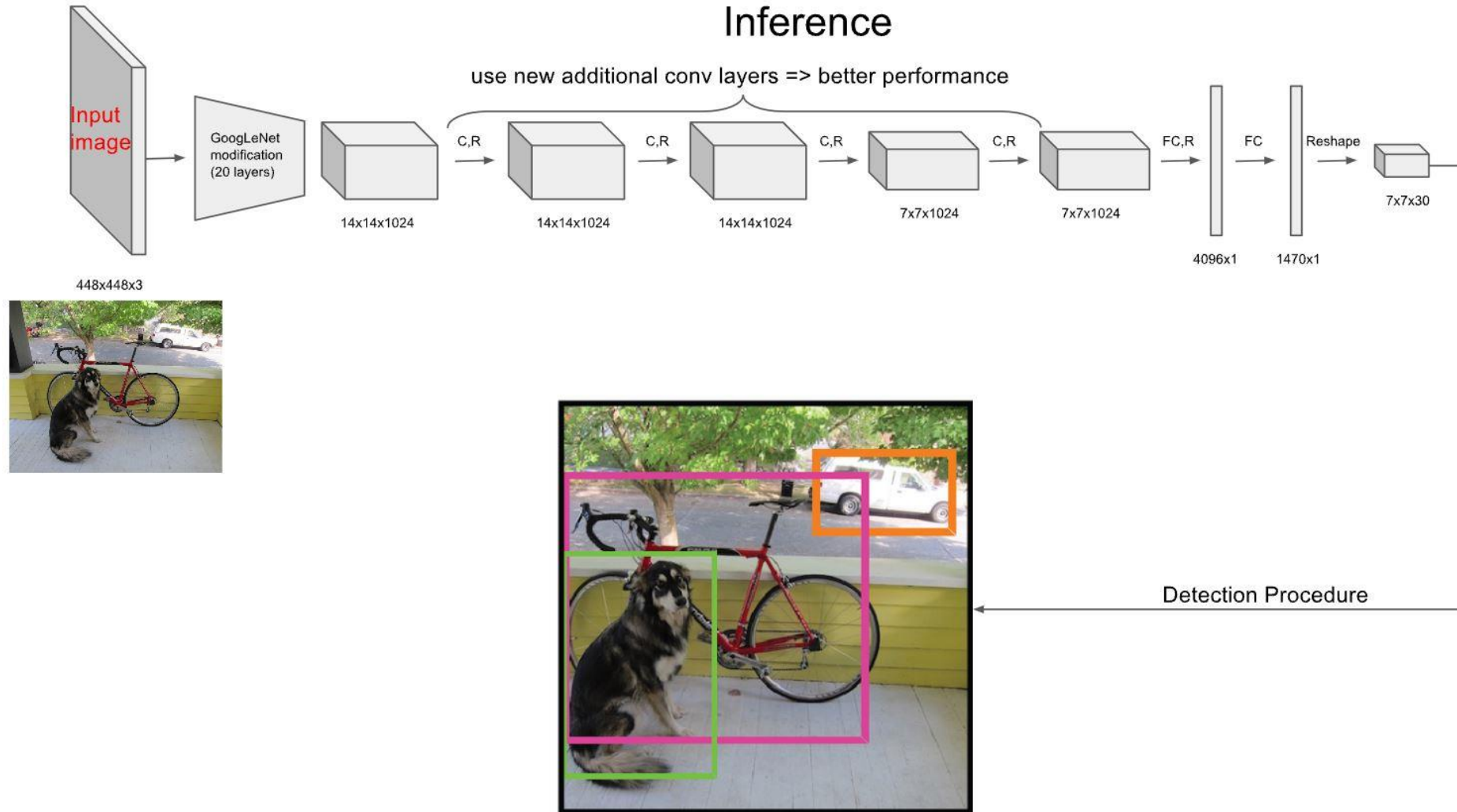
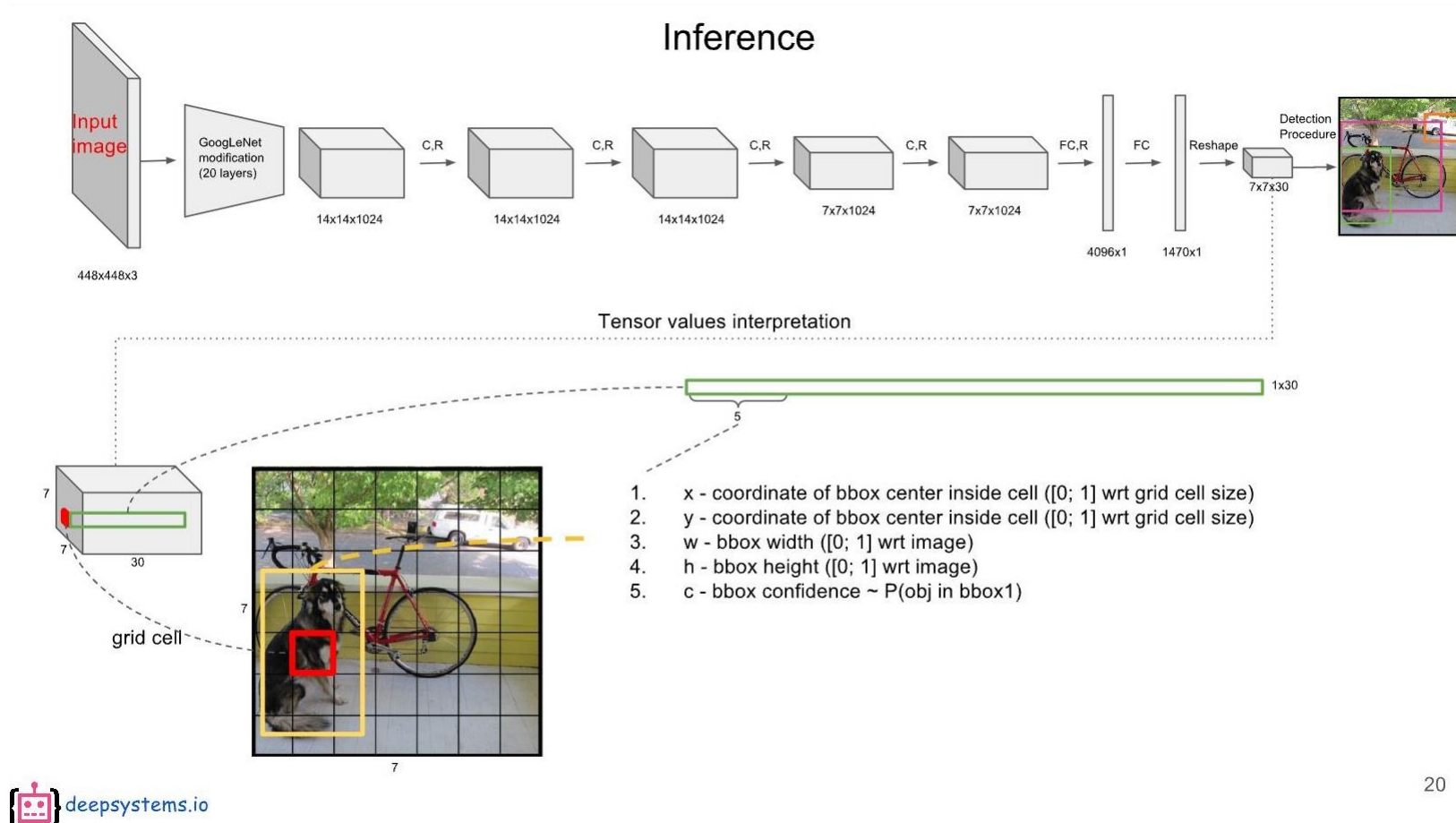


Figure 3: The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

Network Design: Better Visualization !



Inference Process

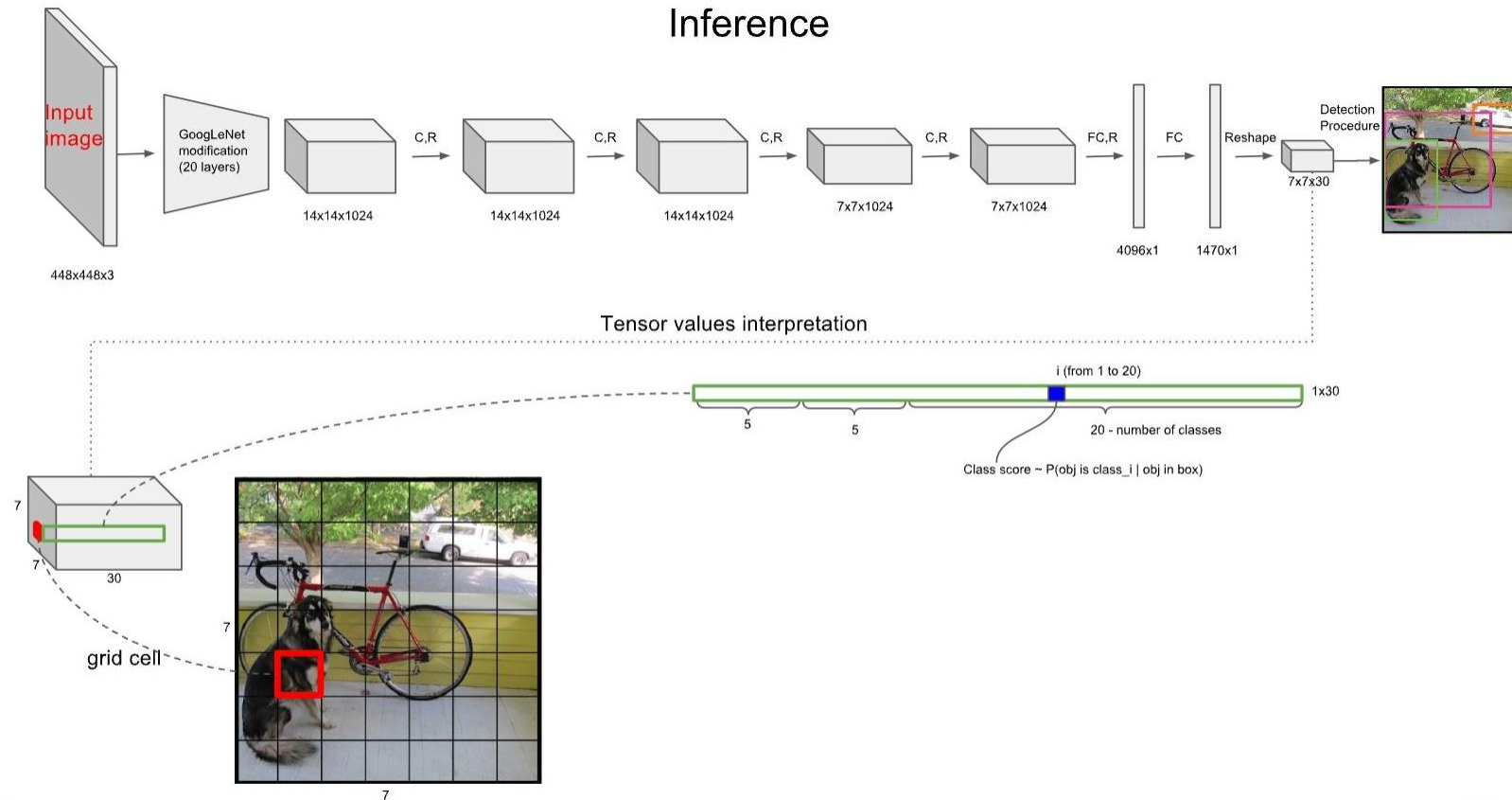


20

Grid cell의 7x7 설정으로 인해 총 49개의 Grid Cell 지님

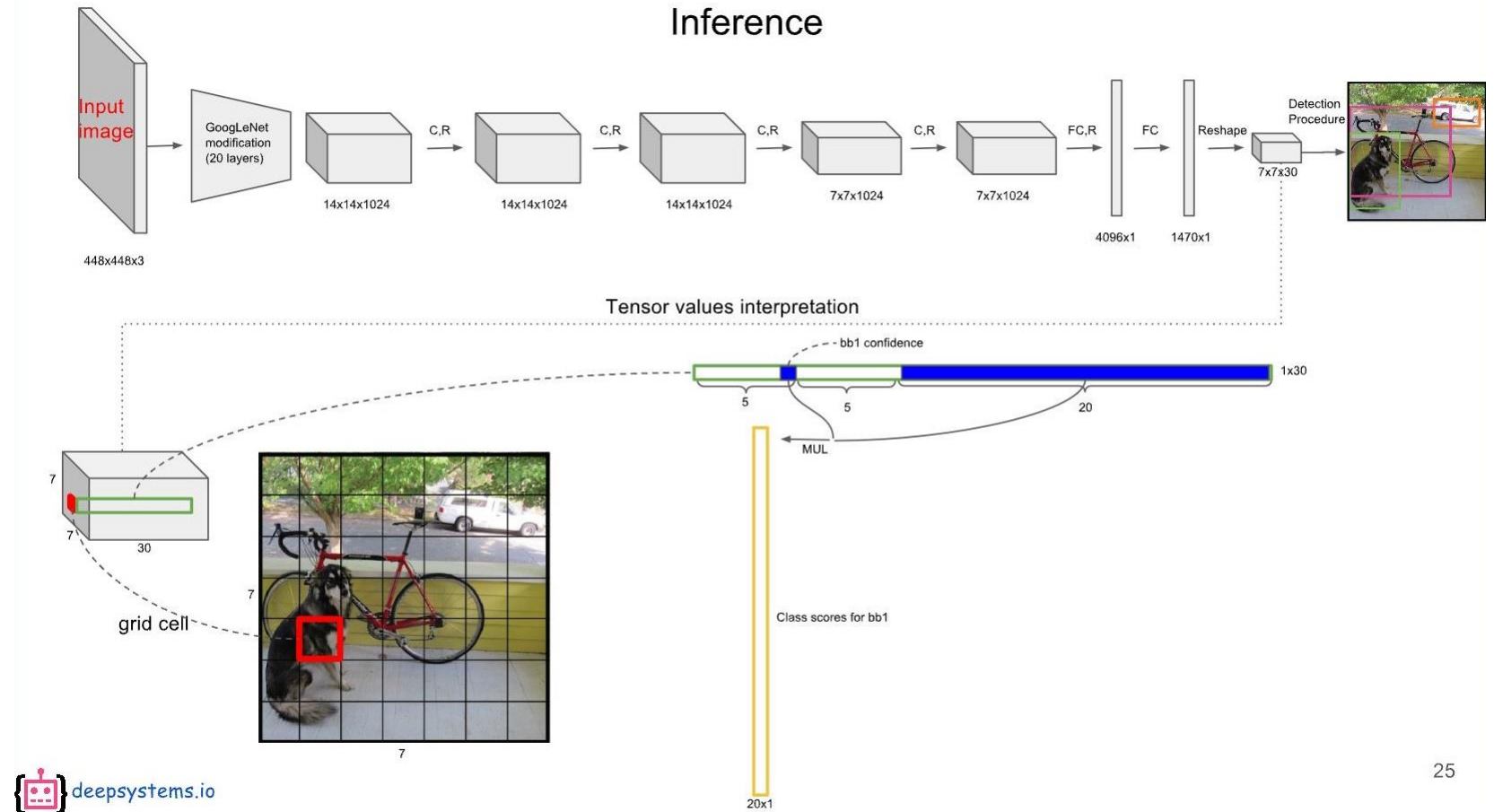
각각의 Grid cell은 B개의 Bounding Box를 지니며, 최초 5개의 값은 Grid Cell은 첫 번째 Bounding box에 대한 값

Inference Process



6-10은 두 번째 Bounding box에 대한 값이며,
나머지 20개의 값은 전체 20개 Class에 대한 Conditional Class probability

Inference Process



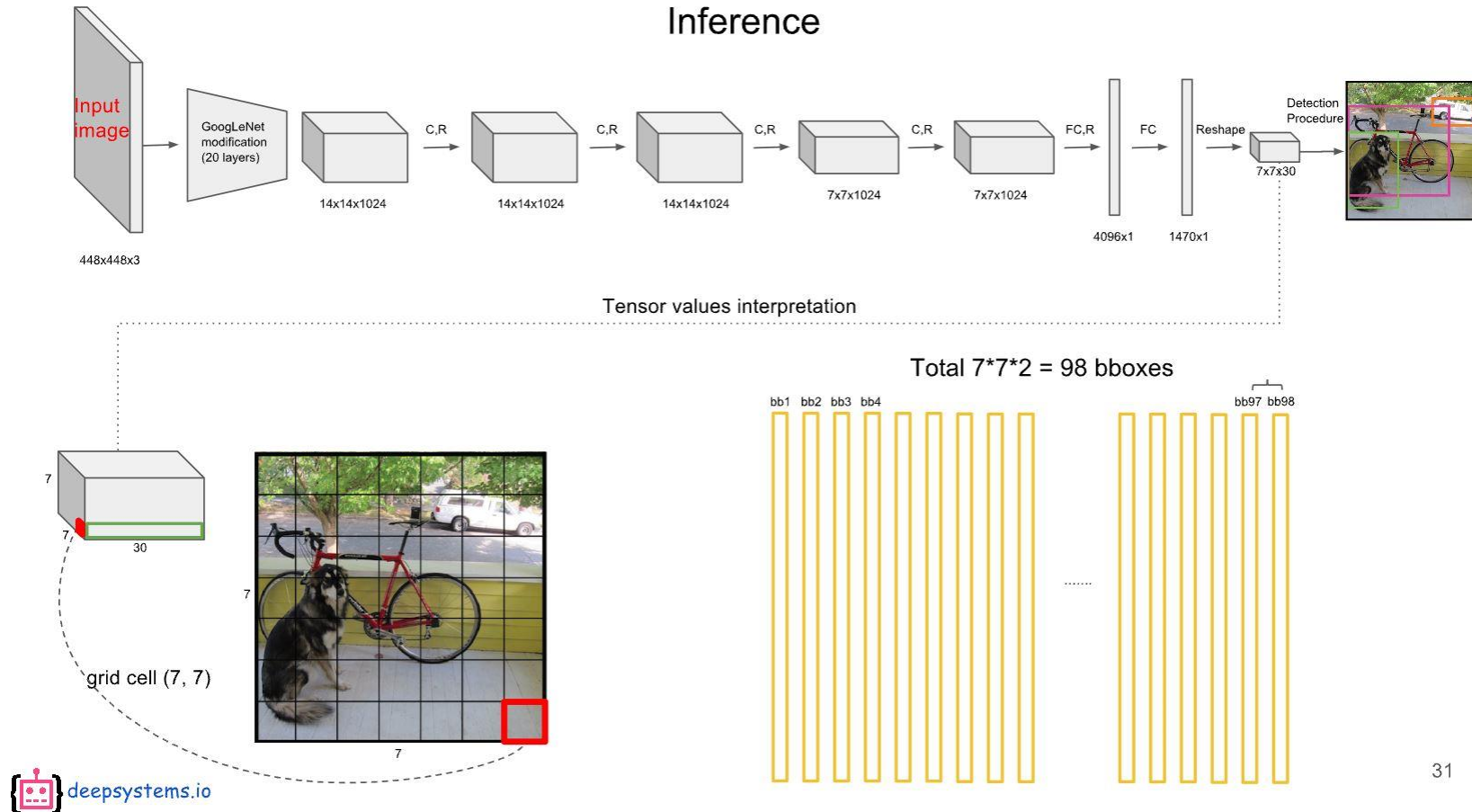
25

첫 번째 Bounding box의 confidence score와 각 Conditional Class Probability를 곱하면

첫 번째 bonding box의 class-specific confidence score가 나옴

마찬가지로 두 번째 bounding box의 class-specific confidence score도 구할 수 있음

Inference Process



31

앞선 계산들로 총 98개의 class-specific confidence score를 얻을 수 있음 !

Training Process

1. Grid Cell의 여러 Bounding box들 중, Ground-truth box와의 IOU가 가장 높은

Bounding box를 **predictor**로 설정

2. 1번의 기준에 따라 우측의 Notation들이 사용

1) Object가 존재하는 Grid cell i 의 predictor bounding box j $\mathbb{1}_{ij}^{obj}$ (1)

2) Object가 존재하지 않는 Grid cell i 의 bounding box j $\mathbb{1}_{ij}^{noobj}$ (2)

3) Object가 존재하는 Grid cell i $\mathbb{1}_i^{obj}$ (3)

Training Process

$$\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \quad (1)$$

$$+ \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \quad (2)$$

$$+ \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left(C_i - \hat{C}_i \right)^2 \quad (3)$$

$$+ \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} \left(C_i - \hat{C}_i \right)^2 \quad (4)$$

$$+ \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \quad (5)$$

(1) Object가 존재하는 grid cell i의 predictor bounding box j에 대해, x와 y의 loss를 계산

(2) Object가 존재하는 grid cell i의 predictor bounding box j에 대해, w와 h의 loss를 계산

큰 box에 대해서는 small deviation을 반영하기 위해 제곱근을 취한 후, sum-squared error (같은 error라도 larger box의 경우 상대적으로 IOU에 영향 적게 줌)

(3) Object가 존재하는 grid cell i의 predictor bounding box j에 대해, confidence score의 loss를 계산 ($C_i = 1$)

(4) Object가 존재하지 않는 grid cell i의 bounding box j에 대해, confidence score의 loss를 계산 ($C_i = 0$)

(5) Object가 존재하는 grid cell i에 대해, conditional class probability의 loss 계산 (Correct class c: $p_i(c)=1$, otherwise: $p_i(c)=0$)

Training Process

$$\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \quad (1)$$

$$+ \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \quad (2)$$

$$+ \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \quad (3)$$

λ_{coord} : coordinates(x,y,w,h)에 대한 loss와 다른 loss들과의 균형을 위한 balancing parameter.

λ_{noobj} : obj가 있는 box와 없는 box간에 균형을 위한 balancing parameter. (일반적으로 image내에는 obj가 있는 cell보다는 obj가 없는 cell이 훨씬 많으므로)

$$+ \sum_{i=0}^{\infty} \mathbb{I}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \quad (5)$$

(1) Object가 존재하는 grid cell i의 predictor bounding box j에 대해, x와 y의 loss를 계산

(2) Object가 존재하는 grid cell i의 predictor bounding box j에 대해, w와 h의 loss를 계산

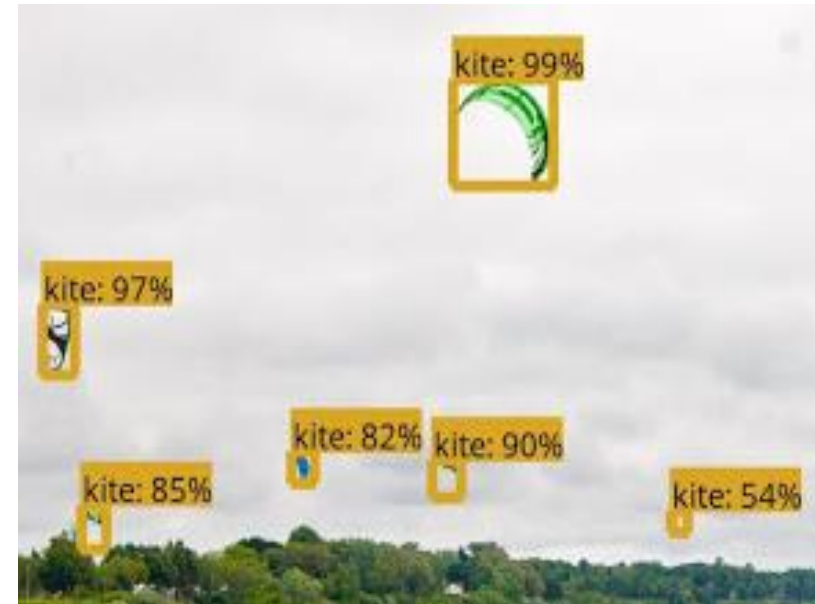
큰 box에 대해서는 small deviation을 반영하기 위해 제곱근을 취한 후, sum-squared error (같은 error라도 larger box의 경우 상대적으로 IOU에 영향 적게 줌)

(3) Object가 존재하는 grid cell i의 predictor bounding box j에 대해, confidence score의 loss를 계산. ($C_i C_i = 1$)

(4) Object가 존재하지 않는 grid cell i의 bounding box j에 대해, confidence score의 loss를 계산. ($C_i C_i = 0$)

(5) Object가 존재하는 grid cell i에 대해, conditional class probability의 loss 계산. (Correct class c: $p_i(c)p_i(c)=1$, otherwise: $p_i(c)p_i(c)=0$)

Limitation of YOLO



1. 각각의 Grid cell이 하나의 Class만을 예측할 수 있으므로, 작은 객체 여러 개가 존재하는 이미지에 대해서는 학습 어려움
2. Bounding box의 형태가 학습 데이터를 통해서만 학습되기 때문에, 새로운 Input으로 이전에 학습되지 않은 형태의 Bounding box를 예측해야 할 경우, 제대로 된 예측을 할 수 없음
3. 몇 단계의 Convolutional Layer를 거쳐 나온 Feature map을 대상으로 Bounding box를 예측하기 때문에 객체의 위치를 정확히 파악하는 Localization이 다소 부정확해지는 경우가 발생

Experiments

Real-Time Detectors	Train	mAP	FPS
100Hz DPM [31]	2007	16.0	100
30Hz DPM [31]	2007	26.1	30
Fast YOLO	2007+2012	52.7	155
YOLO	2007+2012	63.4	45
Less Than Real-Time			
Fastest DPM [38]	2007	30.4	15
R-CNN Minus R [20]	2007	53.5	6
Fast R-CNN [14]	2007+2012	70.0	0.5
Faster R-CNN VGG-16[28]	2007+2012	73.2	7
Faster R-CNN ZF [28]	2007+2012	62.1	18
YOLO VGG-16	2007+2012	66.4	21

Table 1: Real-Time Systems on PASCAL VOC 2007. Comparing the performance and speed of fast detectors. Fast YOLO is the fastest detector on record for PASCAL VOC detection and is still twice as accurate as any other real-time detector. YOLO is 10 mAP more accurate than the fast version while still well above real-time in speed.

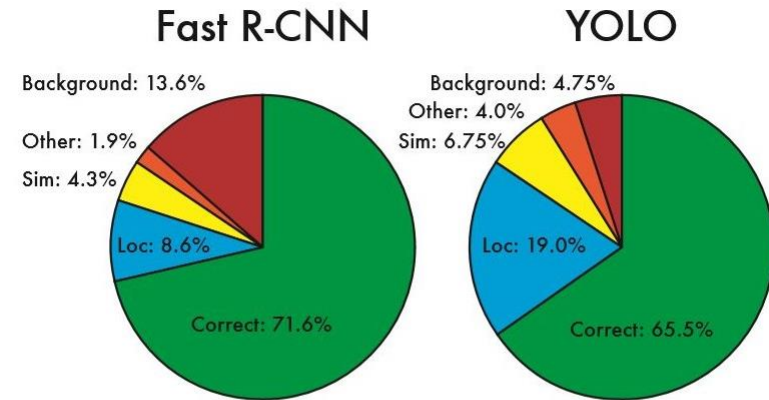
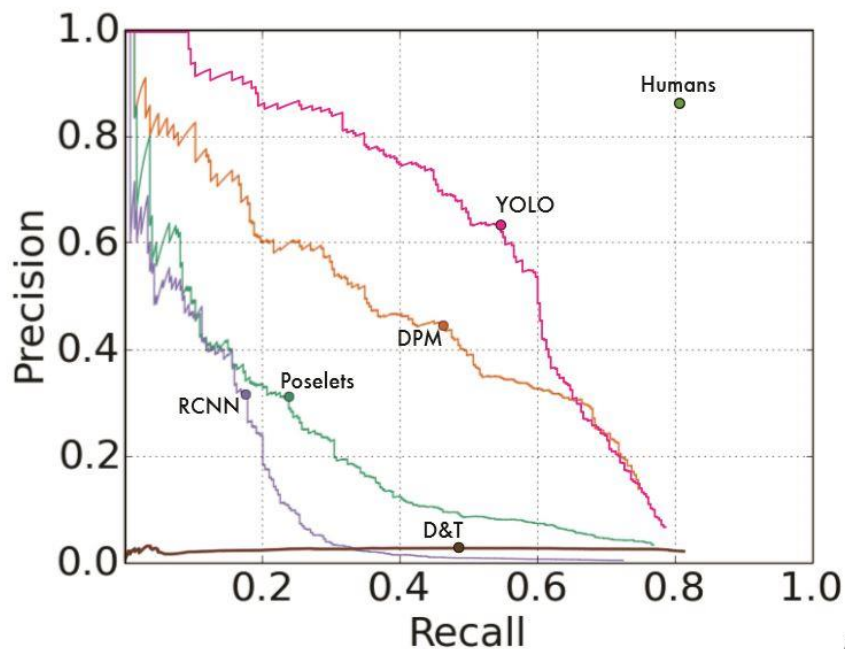


Figure 4: Error Analysis: Fast R-CNN vs. YOLO These charts show the percentage of localization and background errors in the top N detections for various categories (N = # objects in that category).

- 다른 Real-Time Detection 시스템들보다 월등히 높은 mAP를 기록함
- Fast R-CNN과 비교해보았을 때, 훨씬 낮은 Background error를 보이며, 이는 이미지 전체를 보고 학습하기 때문에

배경에 대한 학습이 잘 되었다는 반증이 되기도 함 !

Experiments



(a) Picasso Dataset precision-recall curves.

	VOC 2007 AP	Picasso AP	Picasso Best F_1	People-Art AP
YOLO	59.2	53.3	0.590	45
R-CNN	54.2	10.4	0.226	26
DPM	43.2	37.8	0.458	32
Poselets [2]	36.5	17.8	0.271	
D&T [4]	-	1.9	0.051	

(b) Quantitative results on the VOC 2007, Picasso, and People-Art Datasets. The Picasso Dataset evaluates on both AP and best F_1 score.

Figure 5: Generalization results on Picasso and People-Art datasets.

일반 이미지들로 Pre-train 후, 예술 작품에서 Object를 검출하게 한 결과,

다른 모델들 보다 훨씬 더 좋은 성능을 보였고, 이는 일반화가 아주 잘 되고 있다는 결과!

Conclusion

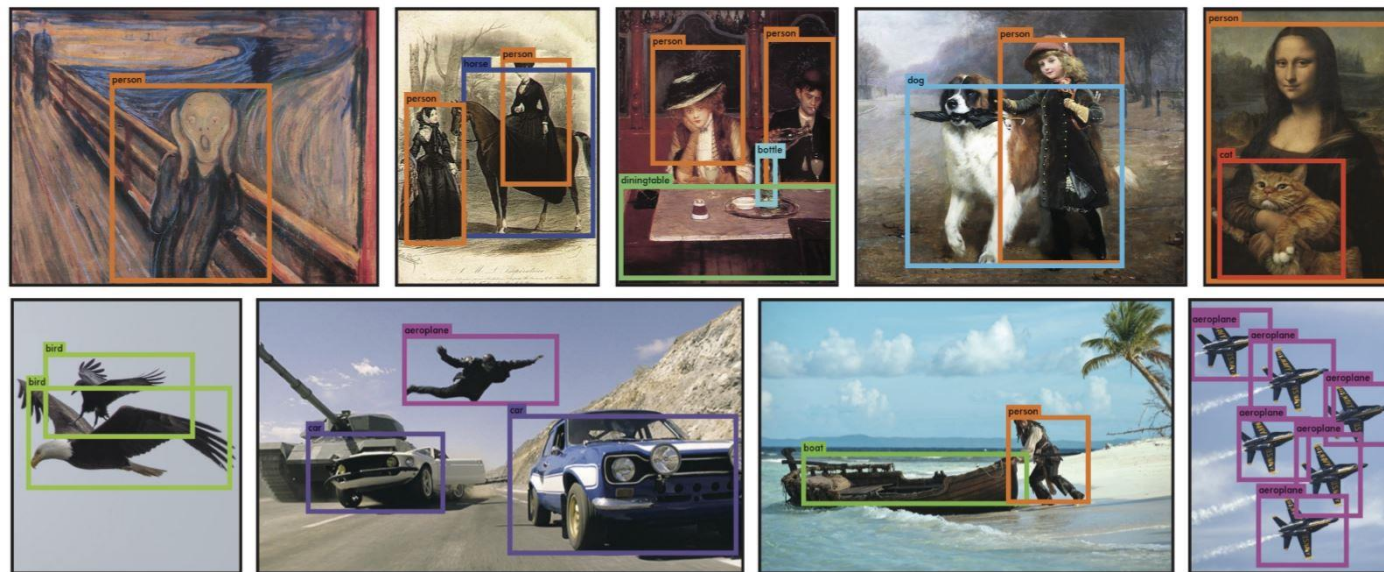


Figure 6: Qualitative Results. YOLO running on sample artwork and natural images from the internet. It is mostly accurate although it does think one person is an airplane.

- 우리의 YOLO 모델은 간단하게 구현이 가능하며, 전체 이미지를 바로 학습시킬 수 있음
- Classified-based 접근 방식과는 다르게 Detection 성능과 직접적인 관계를 맺고 있는 loss function을 통해 학습
- YOLO는 Real-Time Object Detection의 성능 지표를 올렸으며, 일반화가 잘 되기 때문에

새로운 도메인에도 쉽게 적용 가능한 모델이다 !

<EOP>